

This limits the player to three bullets at a time. We'll use this setting in `AlienInvasion` to check how many bullets exist before creating a new bullet in `_fire_bullet()`:

```
alien_invasion.py    def _fire_bullet(self):
                    """Create a new bullet and add it to the bullets group."""
                    if len(self.bullets) < self.settings.bullets_allowed:
                        new_bullet = Bullet(self)
                        self.bullets.add(new_bullet)
```

When the player presses the spacebar, we check the length of bullets. If `len(self.bullets)` is less than three, we create a new bullet. But if three bullets are already active, nothing happens when the spacebar is pressed. When you run the game now, you should only be able to fire bullets in groups of three.

Creating the `_update_bullets()` Method

We want to keep the `AlienInvasion` class reasonably well organized, so now that we've written and checked the bullet management code, we can move it to a separate method. We'll create a new method called `_update_bullets()` and add it just before `_update_screen()`:

```
alien_invasion.py    def _update_bullets(self):
                    """Update position of bullets and get rid of old bullets."""
                    # Update bullet positions.
                    self.bullets.update()

                    # Get rid of bullets that have disappeared.
                    for bullet in self.bullets.copy():
                        if bullet.rect.bottom <= 0:
                            self.bullets.remove(bullet)
```

The code for `_update_bullets()` is cut and pasted from `run_game()`; all we've done here is clarify the comments.

The while loop in `run_game()` looks simple again:

```
alien_invasion.py    while True:
                    self._check_events()
                    self.ship.update()
                    self._update_bullets()
                    self._update_screen()
                    self.clock.tick(60)
```

Now our main loop contains only minimal code, so we can quickly read the method names and understand what's happening in the game. The main loop checks for player input, and then updates the position of the ship and any bullets that have been fired. We then use the updated positions to draw a new screen and tick the clock at the end of each pass through the loop.